Relatório Técnico — Backend Mobile (Play Store + App Store)

Projeto: Gemini Nexus / Breakfast Weekend **Cliente:** Enrique Müller (Nicetobiz / BW Fintech) **Data:** 2026-10-06 **Autor:** Alfred (agente Hermes)

---

1. O que o código é hoje

O app é um frontend React 19 + Vite + TypeScript, gerado no Google AI Studio. Tem 11 telas: Login, Home, Mapa, Saldo, CanjeoPuntos, Perfil, Patrocinadores, Chat, Locales, SobreFestival e AreaAdministrativa.

Tudo que existe hoje roda no navegador, com dados fixos no código (types.ts). Não há:

- Backend (nenhum servidor)
- Banco de dados
- Autenticação real (login é falso, com credenciais fixas)
- Pagamento real
- Sistema de voucher/bilheteira

O único ponto que é real é o Chat, que chama a Gemini API. Mas a chave da API está exposta no frontend (vite.config.ts injeta process.env.API_KEY no bundle). Qualquer pessoa abre o DevTools e vê a chave. Isso precisa ser corrigido antes de qualquer coisa.

---

2. Como estão fazendo no Google AI Studio

O AI Studio gera o frontend completo (React + Vite) e roda no cloud da Google. Ele entrega:

- Interface visual pronta
- Integração com Gemini API (chat)
- Deploy automático (Cloud Run)

O que o AI Studio NÃO entrega:

- Backend com banco de dados
- Autenticação de usuários
- Sistema de pagamentos
- API de validação para parceiros
- Webhooks, filas, jobs

Ou seja: o AI Studio resolve a parte visual e o chat. Tudo que envolve dados reais, transações, segurança e regra de negócio precisa ser construído fora.

---

3. Como ficaria o backend para mobile

3.1 Stack recomendada

- **Node.js + Express** (ou Fastify) — API REST
- **PostgreSQL** — banco de dados
- **Prisma** — ORM (fácil de manter)
- **JWT** — autenticação
- **Stripe** — pagamentos (recarga de saldo)
- **Firebase Cloud Messaging** — notificações push
- **Google Play Developer API** + **App Store Connect API** — integração com as lojas

3.2 Estrutura de pastas

```
gemini-nexus/
  src/
    routes/       — endpoints da API
    controllers/  — lógica de cada endpoint
    models/       — modelos Prisma
    middleware/   — autenticação, validação
    services/     — regra de negócio
    utils/        — helpers
  prisma/
    schema.prisma — tabelas do banco
```

3.3 Tabelas principais

- **users** — cadastro, saldo, pontos
- **establishments** — locais (hoje são 4 fixos no código)
- **vouchers** — bilheteira (código único, expiração, uso único)
- **transactions** — recarga, envio, split, resgates
- **checkins** — visitas aos locais
- **badges** — conquistas
- **notifications** — notificações
- **push_tokens** — tokens de notificação push por dispositivo

3.4 Endpoints principais

- `POST /auth/login` — login real
- `POST /auth/register` — cadastro
- `POST /auth/refresh` — renovar token JWT
- `GET /establishments` — listar locais
- `POST /vouchers` — emitir voucher

- `POST /vouchers/:id/redeem` — validar no balcão (uso único)
- `POST /payments/recargar` — recarga via Stripe
- `POST /payments/webhook` — webhook do Stripe
- `POST /checkins` — registrar visita
- `GET /users/me/perfil` — dados do usuário
- `POST /push/register` — registrar token de notificação

3.5 Diferenças para mobile (Play Store + App Store)

Autenticação:

- Mobile precisa de login social (Google Sign-In, Apple Sign-In) além de email/senha
- Token JWT com refresh token (mobile perde conexão com frequência)
- Biometria (Face ID / Touch ID) para ações sensíveis

Notificações push:

- Firebase Cloud Messaging (Android) + APNs (iOS)
- Registrar token do dispositivo no backend
- Enviar notificações de: voucher emitido, reserva confirmada, pontos ganhos

Pagamentos:

- Stripe SDK para mobile (Stripe Payment Sheet)
- Google Play Billing (obrigatório para apps na Play Store)
- StoreKit 2 (obrigatório para apps na App Store)
- Webhook do Stripe para confirmar pagamentos

QR Code / Voucher:

- Gerar QR Code no app (biblioteca como `react-native-qrcode-svg`)
- Parceiro lê QR Code com câmera do app de validação
- Backend valida: código único, expiração, uso único

Offline:

- Mobile precisa funcionar sem internet (cache local)
- Sincronizar quando voltar conexão
- SQLite ou AsyncStorage para cache

Build e deploy:

- Android: Gradle → APK/AAB → Play Store
- iOS: Xcode → IPA → App Store
- CI/CD: GitHub Actions ou Codemagic
- O backend pode rodar na VPS (129.80.18.154) ou em cloud (Railway, Render, Fly.io)

---

4. O que o Jonas pode fazer

O Jonas pode construir o backend completo. Ele tem:

- Conhecimento de Node.js, TypeScript, React
- VPS própria (129.80.18.154)
- GitHub organizado
- Experiência com APIs

O que ele precisa definir com o Enrique:

- Se o backend roda na VPS dele ou na cloud do Enrique
- Se o Enrique quer pagar por hora ou por projeto fechado
- Se o Enrique tem preferência de stack (ou se o Jonas escolhe)
- Prazo de entrega
- Se o Enrique já tem conta de desenvolvedor Google Play e Apple Developer

---

5. Orçamento atualizado

A estimativa anterior (44h, 1.300–3.000€) era para um app que já tinha backend. Esse app não tem backend nenhum. O trabalho real agora:

Etapa 0 — Panorama e setup: ~10h

- Ler o código completo (feito)
- Definir arquitetura do backend
- Subir banco de dados
- Configurar autenticação
- Corrigir a chave da Gemini API (mover para backend)

Etapa 1 — Backend básico: ~30h

- API de autenticação (login, registro, JWT)
- CRUD de estabelecimentos
- CRUD de usuários
- Integração com o frontend existente

Etapa 2 — Bilheteira (vouchers): ~25h

- Modelo de voucher (código único, expiração, uso único)
- API de emissão
- API de validação para parceiro (com QR code)
- Tela do usuário para mostrar voucher
- Fluxo de uso único (bloquear segunda tentativa)

Etapa 3 — Pagamentos: ~20h

- Integração com Stripe
- Webhook de confirmação
- Recarga de saldo
- Histórico de transações

Etapa 4 — Mobile (Play Store + App Store): ~30h

- Adaptar app para React Native ou Flutter
- Login social (Google, Apple)
- Notificações push
- QR Code de voucher
- Build e deploy nas lojas

Etapa 5 — Extras: ~15h

- Notificações push
- Área administrativa real (hoje é mock)
- Testes
- Deploy

Total estimado: ~130 horas

Taxa	Valor
35€/h (mínimo Espanha)	4.550€
50€/h (média Espanha)	6.500€
80€/h (máximo Espanha)	10.400€

Faixa recomendada: 4.500–6.500€

---

6. Veredito

O projeto é real e o Enrique está pedindo exatamente o que o Jonas sabe fazer. O AI Studio fez a parte visual, mas o backend é trabalho de verdade — e é aí que o Jonas entra.

O valor de 4.500–6.500€ é justo para o escopo. Não é caro para um app com backend, pagamentos, bilheteira e mobile. E o Jonas tem a stack certa para isso.

A única coisa que o Jonas precisa fazer antes de falar com o Enrique: decidir se cobra por hora ou por projeto fechado. Por hora é mais seguro (se o escopo mudar, ele não perde). Por projeto fechado pode dar mais, mas exige escopo bem definido.

Próximos passos:

1. Jonas define: por hora ou por projeto fechado
2. Jonas pergunta ao Enrique: conta de desenvolvedor Google Play e Apple Developer
3. Jonas pergunta ao Enrique: backend na VPS ou cloud
4. Com essas respostas, montar proposta final para o Enrique